

Prof. Dr. Stefan Leutenegger

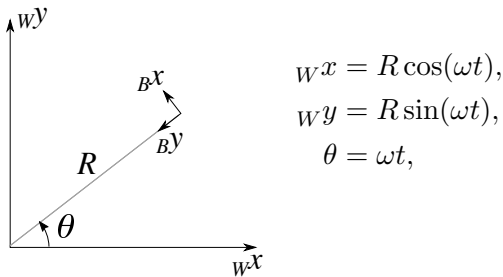
Teaching Assistants: Leonard Freissmuth and Leonard Diederich

Exercise 1: State and Map Representations

Session date: 24/02/2026

Task 1: 2D Lawn-mower Robot

Consider a lawn-mower robot (robot frame $\mathcal{F}_B$) moving around in the $\mathcal{F}_W$ x-y plane in a way parameterised by time as follows:



where θ denotes the robot orientation as indicated in the sketch.

- Write down the homogeneous transformation matrix T_{WB} (for the full 6D pose) as a function of time.
- As alternative forms of representing the orientation, compute the axis-angle and quaternion $\mathbf{q}_{WB}$ as functions of time.
- Now assume that the robot actually is equipped with a magnetometer that measures the magnetic field vector. The magnetic field vector is assumed to be known in the world frame for the specific location where the lawn-mower is deployed, and amounts to ${}_W\mathbf{m}$. Derive a measurement model $\mathbf{z} = \mathbf{h}(\mathbf{x}) + \mathbf{v}$, where $\mathbf{x} = [{}_Wx, {}_Wy, \theta]^T$ and $\mathbf{v}$ denotes noise.

Task 2: Visualising an Aeroplane (Python)

Open the Python3 script `e01_aeroplane.py` in a (virtual) python environment where you have the package `vedo==2025.5.4` installed. There, you will find a visualiser with sliders to set the Tait-Bryan angles ϕ, θ, ψ (roll, pitch, yaw) that describe the orientation of an aeroplane (frame $\mathcal{F}_B$) relative to world frame $\mathcal{F}_E$. The world frame is already being drawn. Now it is your task to finish writing this visualiser. You should follow the convention regarding aeronautics from the lecture, i.e. the world-frame (Earth-fixed, tangential) $\mathcal{F}_E$ should be North-East-Down (NED).

- Familiarise yourself with the code and get the script to run by installing the necessary dependencies (notably `vedo`). It should interactively visualise the aeroplane in 3D with zero roll, pitch, and yaw, and the sliders won't yet have an effect on setting orientation.
- The visualiser should now put the aeroplane (frame $\mathcal{F}_B$) into the right orientation, and also plot its coloured x-y-z axes. Note how the aeroplane mesh was defined in a z-up convention, making an additional transform necessary to flip it around in the end (which is already implemented).

- iii) Now, in addition, there are two cameras (index $i \in 1, 2$) mounted at the wingtips, positioned ${}_B\mathbf{t}_{C_i} = [0, \pm 2, 0]$. They are facing forward (i.e. the camera z-axes are pointing along the $\vec{\mathcal{F}}_B$ x axis) and the camera x-axes point along the y axis of $\vec{\mathcal{F}}_B$. Derive the full transformations $\mathbf{T}_{BC_i}$ and plot the respective camera frame axes into your interactive visualisation, too.

Task 3: Probability Density Functions

Consider the following Gaussian distribution for two continuous random variables $\mathbf{x} = [x_1, x_2]^T$:

$$p(\mathbf{x}) = \frac{1}{2\pi\sqrt{\det \Sigma}} \exp\left(-\frac{1}{2} \left((\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)\right),$$

where the means are $\boldsymbol{\mu} = [\mu_1, \mu_2]^T$ and Σ denotes the 2-by-2 covariance matrix.

- i) Prove that for $\Sigma = \text{diag}[\sigma_1^2, \sigma_2^2]$ with $\sigma_{1,2} \in \mathbb{R}$, x_1 and x_2 are independent. Also compute the respective distributions $p(x_1)$ and $p(x_2)$, further pointing out their means and standard deviations.

- ii) Now compute $p(x_1|x_2 = 0)$ for an arbitrary (positive semi-definite) covariance matrix $\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{12} & \Sigma_{2,2} \end{bmatrix}$.

What are mean and standard deviation?

Task 4: Balloons in Cappadocia (Blender)



Figure 1: Balloons in Cappadocia, Image by Robert_K on Pixabay:
<https://pixabay.com/photos/cappadocia-hot-air-balloons-turkey-6771879/>

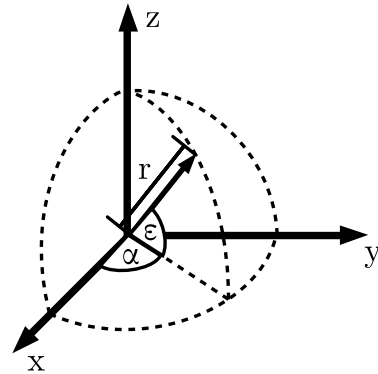


Figure 2: Polar coordinates used in this exercise.

You are on a hot-air balloon in Cappadocia and want to film your friend, who skydives from a second hot-air balloon. You use a camera mounted on a gimbal rigidly attached to the balloon basket. Both balloons translate and rotate during the flight.

Let $\vec{\mathcal{F}}_W$ denote the world frame. The body frames of the balloons are $\vec{\mathcal{F}}_{B_1}$ (filming balloon) and $\vec{\mathcal{F}}_{B_2}$ (jump balloon). Their poses with respect to the world frame are provided by GNSS and magnetometer measurements as homogeneous transforms:

$$\mathbf{T}_{WB_i} \in SE(3), \quad i \in \{1, 2\}.$$

The skydiver cannot carry a GNSS and streaming streaming setup, but instead wears a passive radar reflector. Balloon 2 carries a very accurate and long-range radar that measures the relative position of the skydiver in polar coordinates expressed in $\vec{\mathcal{F}}_{B_2}$ (See Fig. 2)

The camera is rigidly mounted on balloon 1 at a fixed translation

$${}_{B_1}\mathbf{t}_{B_1C} = [0, -4, -19]^T,$$

with camera frame $\underline{\mathcal{F}}_C$ defined as follows:

- z_C points along the viewing direction,
- x_C points to the right in the image,
- y_C points down in the image.

In this exercise, you will use Blender, which can be downloaded from <https://blender.org>. You will receive a Blender scene file, which has been tested on version 4.5.7 LTS and contains:

- two objects representing the balloon frames $\underline{\mathcal{F}}_{B_1}$ and $\underline{\mathcal{F}}_{B_2}$,
- an animated skydiver object,
- an animated camera that zooms towards the skydiver,
- an object representing the camera pose T_{WC} to be updated each frame.

To see the console output of your blender script, you need to start blender from the terminal if you are on Linux. In windows, you can show the console output through the menu ribbon and going to Window → Toggle System Console. The viewports show a 3D view on the top left, the camera view on the bottom left and a code editor on the right. In Blender, use:

- **Middle mouse drag** to orbit the view (or click and drag the 3D gizmo on the top right of the 3D view port if you are on a laptop),
- **Shift + middle mouse drag** to pan (or use the hand icon below the gizmo if you are on a laptop),
- **Scroll wheel** to zoom (or a trackpad scroll motion if you are on a laptop),
- **Space bar** (while hovering the mouse over the 3D view port) to play and pause the animation.

The exercise is implemented as a Python *frame change handler* that runs once per animation frame. Your task is to complete the computations so that the camera continuously points at the skydiver throughout the animation. To register the frame change handler, execute the script with the triangle button at the top ribbon of the text editor. When you then play back the animation with the spacebar, you'll see your code in action.

1. Convert the radar polar measurement (r, α, ε) to Cartesian translation in $\underline{\mathcal{F}}_{B_2}$.

$${}_{B_2}\mathbf{t}_{B_2S} = [{}_{B_2}x(r, \alpha, \varepsilon), {}_{B_2}y(r, \alpha, \varepsilon), {}_{B_2}z(r, \alpha, \varepsilon)]^T$$

2. Compute the camera rotation with respect to $\underline{\mathcal{F}}_{B_1}$ such that it points at the skydiver. Using the provided T_{WB_1} , T_{WB_2} , and ${}_{B_1}\mathbf{t}_{B_1C}$ as well as the radar-derived ${}_{B_2}\mathbf{t}_{B_2S}$, compute the skydiver position expressed in $\underline{\mathcal{F}}_C$. Afterwards, compute the camera's rotation matrix $\mathbf{R}_{B_1C}$ such that:

- The z -axis of $\underline{\mathcal{F}}_C$, i.e. the viewing direction, aligns with ${}_C\mathbf{t}_{CS}$,
- The x -axis of $\underline{\mathcal{F}}_C$ aligns with the local horizon, i.e. lies in xy -plane of $\underline{\mathcal{F}}_W$.

(Handle the degenerate case where the viewing direction is parallel to “down”.)

3. Update T_{WC} by assembling $\mathbf{R}_{B_1C}$ and ${}_{B_1}\mathbf{t}_{B_1C}$ into a homogeneous transform and projecting it into world frame.